set :

1.set and dic both defined as { }

2.only int,float and alphabaet can be sorted(increasing order) as out put

other wise set defined as unordered collection of iteam

3. set is immutable thats why indexing & slicing are not allowed

In [1]:
```
s1= set() #empty set
s1
```

Out[1]:
```
set()
```

In [2]:
```
type(s1)
```

Out[2]:
```
set
```

In [3]:
```
s2= {}   #empty set
s2
```

Out[3]:
```
{}
```

In [4]:
```
type(s2)
```

Out[4]:
```
dict
```

In [5]:
```
s3 = {20,90,66,28,55,17}  #int sorted
s3
```

Out[5]:
```
{17, 20, 28, 55, 66, 90}
```

In [6]:
```
s4 = {'z','p','a','r','c'} #alphabed sorted
s4
```

Out[6]:
```
{'a', 'c', 'p', 'r', 'z'}
```

In [7]:
```
s5 = {10,10.50,'nit',2+5j,(2,5,9),True} #mix data type gives unordered collectio
s5
```

Out[7]:
```
{(2+5j), (2, 5, 9), 10, 10.5, True, 'nit'}
```

In [8]:
```
s5={[1,2,3]}   #Set elements are immutable (cannot be changed).
s5
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[8], line 1
----> 1 s5={[1,2,3]}   #Set elements are immutable (cannot be changed).
      2 s5

TypeError: unhashable type: 'list'
```

In [ ]:
```
s6={10,10,25,50,90,90,15}  #duplicate not allowed
s6
```

In [ ]:
```
s3
```

In [ ]:
```
s3[0]    # set not allowed indexing due to unordered collection of iteam
```

set method

In [ ]:
```
s3
```

In [ ]:
```
s3.add(100)
s3
```

In [ ]:
```
s3.discard(110) #remove element from set if the element does not exist in the se
s3
```

In [ ]:
```
s3.discard(17)
s3
```

In [ ]:
```
s3
```

In [ ]:
```
s3.remove(28)
s3
```

In [ ]:
```
s3.remove(99) #remove element from set if the element does not exist it will thr
```

In [ ]:
```
s3
```

In [ ]:
```
s3.pop()
s3
```

In [ ]:
```
myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight'}
myset
```

In [ ]:
```
myset_new = myset   # Create a new reference "myset_new"
myset_new
```

In [ ]:
```
id(myset), id(myset_new)   # The address of both myset & myset_new will be the s
```

In [ ]:
```
myset_new
```

In [ ]:
```
myset_new.add('nine')
myset_new
```

In [ ]:
```
myset   # myset will be also impacted as it is pointing to the same Set
```

set operation

1.union (|= pipe)

In [ ]:
```
A = {1,2,3,4,5}   # Union of A and B (All elements from both sets. NO DUPLICATES
B = {4,5,6,7,8}
C = {8,9,10}
A|B
```

In [ ]:
```
A|C
```

In [ ]:
```
A.union(B)
```

In [ ]:
```
A.union(B,C)
```

2.intersection (& = and)

In [ ]:
```
A = {1,2,3,4,5}
B = {4,5,6,7,8}
```

In [ ]:
```
A&B  # Intersection of A and B (Common items in both sets)
      # A.intersection(B)
```

3.Difference

In [ ]:
```

A = {1,2,3,4,5}
B = {4,5,6,7,8}
```

In [ ]:
```
A-B    # set of elements that are only in A but not in B
        # A.difference(B)
```

4.symmetric diffrence

In [ ]:
```
A = {1,2,3,4,5}
B = {4,5,6,7,8} # Symmetric difference (Set of elements in A and B but not in bo
```

In [ ]:
```
A^B
```

Subset , Superset & Disjoint

In [ ]:
```
A = {1,2,3,4,5,6,7,8,9}
B = {3,4,5,6,7,8}
C = {10,20,30,40}
```

In [ ]:
```
A.issuperset(B)
```

In [ ]:
```
B.issubset(A)
```

In [ ]:
```
A.isdisjoint(B)
```

In [ ]:
```
A.isdisjoint(C)
```

looping in set

In [10]:
```
myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight'}
myset
```

Out[10]:
```
{'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

In [11]:
```
for i in myset:
    print(i)
```

```
six
four
eight
seven
one
five
two
three
```

In [12]:
```
for i in enumerate(myset):
    print(i)
```

```
(0, 'six')
(1, 'four')
(2, 'eight')
(3, 'seven')
(4, 'one')
(5, 'five')
(6, 'two')
(7, 'three')
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: